

Senior Automation Engineering Proposal

Hello,

My name is Fabricio Santana. I am a software engineer and technical lead with 20+ years of experience in backend development, software architecture, database integration, automated testing, cloud computing, and delivery of critical systems. I have also led large engineering teams and worked on projects where data consistency, business rules, validation, security, and maintainability are essential.

Your project is a strong match for my background because the main challenge is not only writing automation scripts. The important part is defining the right automation architecture, integrating it into delivery workflows, and making sure the resulting system is reliable, maintainable, and aligned with the real business objective.

My relevant approach for this kind of project is to combine automation architecture, test strategy, backend integration, CI/CD hardening, observability, deployment support, and technical handoff into one coherent delivery plan. This is especially important here because the opportunity mixes QA automation language with references to Shopee, JD, and scraping tools, which may imply a broader or different scope than a standard test-automation role.

Working methodology

My delivery method is designed to reduce risk before development starts. The work begins with a fixed Discovery Sprint and then moves into a delivery package only after scope, priorities, responsibilities, dependencies, and acceptance criteria are clear.

The first step is a one-week Discovery Sprint with a fixed price of USD 500. In this phase, I can hold up to five remote meetings to understand the business problem, review the current platform, analyze the current data sources and workflows, map requirements, identify risks, review integrations, and define priorities. The deliverable is a practical work plan for the implementation, including recommended scope, technical approach, milestones, success criteria, and the most appropriate delivery package.

After discovery, the project can move into one of three fixed-price, four-week delivery packages. The package is selected according to scope, complexity, delivery speed, integration needs, quality requirements, and the amount of engineering support required. Some roles may participate part-time depending on the agreed scope, especially Scrum Master, technical leadership, test automation, DevOps, observability, and deployment support.

Delivery package options

- **Essential Delivery** — USD 2,000, 4 weeks. Includes a Scrum Master and one full-stack engineer. This package is designed for a focused module, MVP, backend feature, import flow, or small integration with limited uncertainty. It is suitable when the scope is well defined, the number of integrations is limited, and the main goal is to deliver a reliable working feature without unnecessary overhead.
- **Product Acceleration** — USD 4,000, 4 weeks. Includes a Scrum Master, two full-stack engineers, and one engineer focused on test automation, DevOps, observability, and deployment. This package is recommended when faster delivery is needed or when the scope includes a product, API, integration, internal workflow, multiple features, or more demanding quality needs.

- **Scale & Intelligence** — USD 6,000, 4 weeks. Includes a Technical lead, Scrum Master, two full-stack engineers, and one engineer focused on test automation, DevOps, observability, and deployment. This package is designed for complex systems, advanced data workflows, automation, auditability, security, observability, broader platform evolution, or stronger technical scale planning.

Role responsibilities are clear: the Scrum Master organizes scope, communication, ceremonies, progress tracking, and delivery alignment; full-stack engineers implement backend, frontend, API, and database changes as needed; the test automation, DevOps, observability, and deployment engineer supports automated tests, CI/CD, monitoring, release quality, and deployment; and the Technical lead, included in the Scale & Intelligence package, owns architecture, technical decisions, code quality, and risk management for more complex projects.

For an automation initiative like this, the implementation setup can include framework design, pipeline integration, environment strategy, observability, reporting, deployment support, and handoff work within the agreed package scope.

Each delivery cycle is executed with Scrum-inspired practices, frequent alignment, technical review, automated testing when applicable, and a demonstration of what was delivered. The client keeps ownership of the produced code, and the scope is agreed before execution to avoid open-ended work and unclear expectations.

Construction defect warranty. After delivery and acceptance, I include a 30-day warranty for construction defects related to the approved scope. This covers fixes for implementation errors, broken agreed behavior, or defects introduced by the delivered code. It does not cover new features, changes in business rules, new target platforms, new scraping flows, or expanded automation coverage not included in the approved scope, third-party service changes, infrastructure changes outside the project, or requests that expand the original acceptance criteria.

Opportunity analysis

This project should be treated as an automation-architecture definition problem, not as a simple scripting task. The opportunity describes QA automation for web, mobile, API, CI/CD, and defect reporting, but it also references Shopee, JD, and powerful scraping tools. That introduces an important scope question: are these target systems under test, targets for data extraction, or both?

This distinction matters because the business value, architecture, compliance risk, and maintenance burden are very different. QA automation aims to reduce regression risk, improve release confidence, increase coverage, and strengthen software quality. Scraping automation aims to create reliable data-extraction workflows, resilience against platform changes, and operational stability in interactions with external platforms.

The main technical risks are selecting the wrong automation architecture, mixing test-automation goals with scraping goals inside one delivery scope, underestimating CI/CD and observability needs, and failing to define a clear maintenance model. If Shopee and JD are part of a scraping requirement, there may also be compliance and platform-policy considerations that should be understood before implementation starts. I would keep any data-extraction work within authorized data access, permitted APIs, compliant browser automation, and the platform-policy boundaries agreed during discovery.

For this reason, I recommend starting with a Discovery Sprint to separate the true objective, identify the correct tooling, confirm supported platforms and environments, and define a bounded

first implementation cycle.

Proposed work plan

I would approach the project in structured phases. The Discovery Sprint is the immediate recommended commitment, and implementation can move directly into a fixed delivery cycle once the automation objective, architecture, and acceptance criteria are confirmed. If the client needs an embedded senior automation profile rather than a fixed delivery package, discovery can also define a part-time support model, cadence, responsibilities, and success metrics.

1. Discovery, scope clarification, and technical design

- Review the current systems, target platforms, supported environments, testing workflow, and business objective behind the automation request.
- Determine whether the first priority is QA automation, scraping automation, or the separation between both.
- Define the preferred automation stack, CI/CD integration model, observability needs, and reporting requirements.
- Produce a concrete implementation plan with automation boundary, recommended architecture, acceptance criteria, responsibilities, dependencies, risks, and package recommendation.

2. Core automation architecture and framework foundation

- Design and implement the automation foundation for the agreed domain, whether test automation, scraping automation, or a clearly separated hybrid.
- Define reusable structure, naming, data handling, selector or contract strategy, reporting, and execution flows.
- Add validation, structured error handling, traceability, and maintainability controls.
- Establish the technical base for pipeline integration, monitoring, and secure environment configuration.

3. Automation workflows and target coverage

- Implement the first agreed automation coverage for web, mobile, API, target-platform flows, or extraction workflows according to the confirmed objective.
- For QA automation, prioritize concrete deliverables such as smoke suites, regression suites, API contract tests, CI quality gates, test reports, flaky-test handling, and defect-reporting flow.
- Keep the first implementation cycle focused on a bounded automation slice rather than broad uncontrolled coverage.
- Structure the logic so it can be maintained and expanded safely after the first release.
- Support reporting and execution feedback according to the agreed operational needs.

4. CI/CD integration, monitoring, and operational hardening

- Integrate the automation into the agreed CI/CD flow when applicable.
- Add execution visibility, logs, failure diagnostics, and operational monitoring.

- Define fallback behavior for flaky environments, unstable targets, or unsupported paths.
- Prepare the automation for team usage and ongoing maintenance.

5. Testing, documentation, deployment support, and handoff

- Test the framework behavior, core automation flows, integrations, and edge cases.
- Document the architecture, execution model, limitations, and maintenance steps.
- Provide deployment support and a handoff session or written notes so your team can operate and extend the framework safely.
- Leave a clear roadmap for the next automation increments when needed.

Estimated timeline

For the immediate next step, my recommendation is a 1-week Discovery Sprint. After discovery, the first implementation cycle can move into a fixed 4-week delivery package, or the engagement can be structured as part-time senior automation support if that better matches the client's team model. For a bounded automation architecture covering QA automation with CI/CD integration, the likely recommendation is one Product Acceleration cycle. If the scope includes compliant data-extraction resilience, broader observability, or hybrid QA plus scraping automation, the likely recommendation is one Scale & Intelligence cycle.

A practical schedule would be:

- Week 1: Discovery Sprint, automation-boundary definition, target-platform review, stack recommendation, launch-scope backlog, acceptance criteria, and implementation plan.
- Weeks 2 to 5: first fixed implementation cycle, covering the agreed automation architecture and initial delivery scope, or a defined part-time senior support period if that model is selected after discovery.

If discovery shows that the first version is narrower than currently implied, the implementation may fit the smaller package or a reduced delivery shape. If it shows broader scraping resilience, deeper mobile automation, larger CI/CD hardening needs, or extensive multi-platform support, I will recommend the larger package or additional cycles accordingly.

Availability and communication

I can work remotely and keep communication in English through Workana messages, scheduled calls, and written progress updates. I usually prefer short alignment checkpoints during the week, especially after reviewing the current workflow and after each major implementation milestone.

This helps reduce misunderstandings and keeps the project aligned with your actual automation goals, engineering workflow, and operational constraints.

Budget

Based on the current understanding, my recommended immediate starting budget is USD 500 for the one-week Discovery Sprint. This gives you a concrete plan before committing to the full

implementation budget, while keeping a clear path to delivery in the following implementation cycle.

After discovery, the first implementation cycle will likely fit one of these packages:

- **Product Acceleration** at USD 4,000 for 4 weeks if the scope is a bounded automation framework with CI/CD integration and moderate platform complexity.
- **Scale & Intelligence** at USD 6,000 for 4 weeks if the scope includes resilient compliant data-extraction workflows, stronger observability, hybrid automation goals, or broader platform complexity.

The Discovery Sprint will refine the scope and confirm the correct implementation package or support model. If the review shows a smaller and tightly bounded automation workflow, I can recommend a narrower delivery shape. If it shows broader platform complexity, compliant data-extraction maintenance needs, stronger reliability requirements, or additional integrations, I will recommend the larger package or additional cycles.

Questions before final confirmation

Before confirming the final scope, I would like to clarify a few points:

1. Is the core objective QA automation, scraping automation, or both?
2. Are Shopee and JD the systems under test, the systems being scraped, or both?
3. What programming language and automation stack are currently in use?
4. Which web, mobile, and API systems must be automated in the first version?
5. Is mobile automation already set up with Appium, Maestro, Detox, or another framework?
6. What does “Powerful Scraping Tools” mean in practice: browser automation, extraction pipelines, monitoring, proxy handling, or another setup?
7. Should the engagement be a fixed implementation cycle or ongoing part-time senior automation support?
8. What CI/CD platform is currently used?
9. What are the main success metrics: lower regression risk, faster releases, higher test coverage, better data-collection reliability, or another outcome?
10. Are there compliance, legal, or platform-policy constraints around Shopee and JD automation?
11. Is there already an approved budget range and a target date for the first operational version?

Once these points are clear, I can confirm the final delivery plan and milestones.